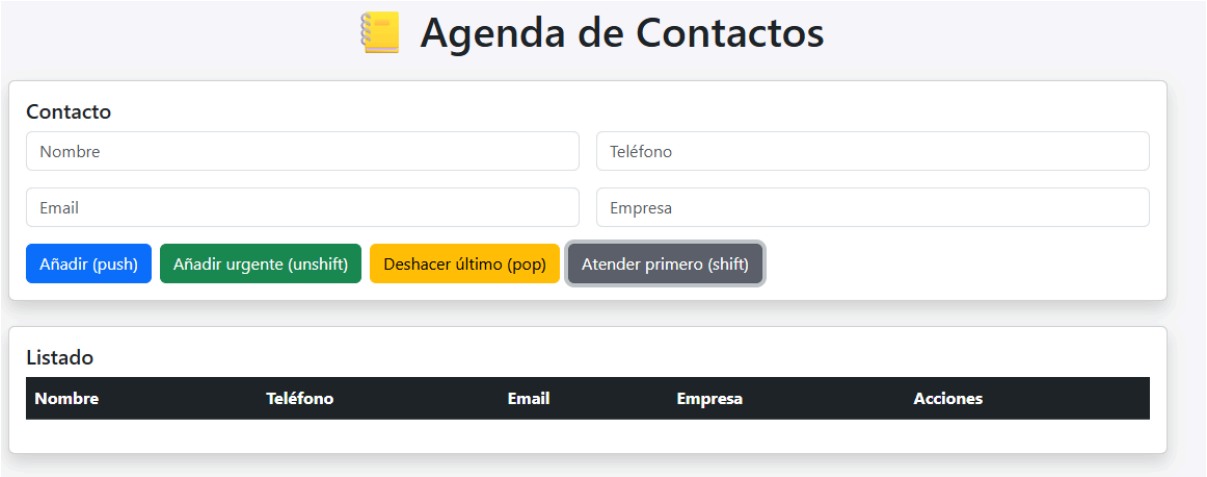
0403ED_01_
Gestión Agenda Contactos



Agenda de Contactos (Arrays + Objetos + CRUD)

🧠 CONTEXTO GENERAL

En aplicaciones reales **no trabajamos con datos sueltos**, sino con:

- 📦 **listas** → arrays
- 🎒 **entidades completas** → objetos

👉 En este ejercicio construiremos una **agenda profesional**, similar a:

- un CRM
- un panel admin
- una app de gestión

0403ED_01_

Gestión Agenda Contactos

🟩 PASO 1 · Modelar los datos (LO MÁS IMPORTANTE)

🎯 Objetivo

Definir **cómo será un contacto** antes de programar.

🧠 Explicación

Un contacto es **una entidad**, no un valor suelto.
Por eso usamos un **objeto**.

```
{
  id: number,
  nombre: string,
  telefono: string,
  email: string,
  empresa: string
}
```

👉 Cada contacto tendrá **todas sus propiedades juntas**.

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 2 · Crear el array de contactos

🎯 Objetivo

Guardar **muchos contactos** en una sola estructura.

🧠 Explicación

Una agenda es una **colección de contactos**, así que usamos un **array de objetos**.

```
const agenda = [
  {
    id: 1,
    nombre: "Ana López",
    telefono: "600123123",
    email: "ana@mail.com",
    empresa: "TechSoft"
  }
];
```

✔ Qué aprendemos

- Un **array** guarda listas
- Un **objeto** describe una entidad completa
- Esta es la estructura usada en **APIs y JSON**

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 3 · Mostrar los contactos (READ)

🎯 Objetivo

Recorrer el array y **pintar los datos en pantalla**.

🧠 Método usado

`forEach()`

```
agenda.forEach(contacto => {
  console.log(contacto.nombre);
});
```

🧠 Por qué `forEach`

- No queremos transformar datos
- Solo ejecutar una acción por cada elemento

✔ Qué aprendemos

- Recorrer arrays de objetos
- Acceder a propiedades: `contacto.nombre`

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 4 · Crear un contacto (CREATE con push)

🎯 Objetivo

Añadir un contacto **al final** de la agenda.

🧠 Método usado

`push()`

```
agenda.push(contacto);
```

🧠 Qué hace push

📦 Añade un elemento **al final del array**

✔ Uso real

- Altas normales
- Registros cronológicos
- Logs

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 5 · Crear un contacto urgente (unshift)

🎯 Objetivo

Añadir un contacto **prioritario**.

🧠 Método usado

`unshift()`

```
agenda.unshift(contacto);
```

🧠 Qué hace `unshift`

📦 Añade un elemento **al inicio del array**

✔ Uso real

- Prioridades
- Soporte urgente
- Tareas críticas

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 6 · Deshacer el último contacto (pop)

🎯 Objetivo

Eliminar el **último contacto añadido**.

🧠 Método usado

`pop()`

```
agenda.pop();
```

🧠 Qué hace pop

📦 Elimina el **último elemento**

✔ Uso real

- Deshacer acciones
- Historial
- Undo básico

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 7 · Atender el primer contacto (shift)

🎯 Objetivo

Eliminar el **contacto más antiguo**.

🧠 Método usado

`shift()`

```
agenda.shift();
```

🧠 Qué hace `shift`

📦 Elimina el **primer elemento**

✔ Uso real

- Colas (FIFO)
- Atención por orden
- Procesos secuenciales

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 8 · Eliminar un contacto concreto (findIndex)

🎯 Objetivo

Eliminar un contacto **por su id**.

🧠 Métodos usados

- `findIndex()`
- `splice()`

```
const index = agenda.findIndex(c => c.id === id);
agenda.splice(index, 1);
```

🧠 Explicación

1. Buscamos la posición del contacto
2. Eliminamos exactamente ese elemento

✔ Qué aprendemos

- Búsqueda en arrays de objetos
- Eliminación controlada
- Uso de `id` como identificador real

0403ED_01_
Gestión Agenda Contactos

🟩 PASO 9 · Crear objetos desde formularios

🎯 Objetivo

Convertir datos del usuario en **objetos JavaScript**.

```
return {
  id: Date.now(),
  nombre,
  telefono,
  email,
  empresa
};
```

🧠 Qué aprendemos

- Objetos literales
- Datos dinámicos
- Simulación de backend

🟩 PASO 10 · Validar datos antes de usarlos

🎯 Objetivo

Evitar datos incompletos.

```
if (!nombre || !telefono || !email || !empresa) {
  alert("Todos los campos son obligatorios");
}
```

✔ Qué aprendemos

- Programación defensiva
- Control de errores
- Lógica real de formularios

0403ED_01_

Gestión Agenda Contactos

🟦 RESUMEN FINAL (CLAVE PARA EXAMEN / ENTREVISTA)

✔ Arrays

- `push`
- `pop`
- `unshift`
- `shift`
- `forEach`
- `findIndex`

✔ Objetos

- Propiedades
- Creación dinámica
- Uso de `id`
- Acceso a datos

✔ Conceptos profesionales

- Modelado de datos
- CRUD real
- Flujo frontend
- Lógica de negocio

🎯 Frase clave para el alumno

> **“No uso métodos porque existan,**
> **los uso porque resuelven un problema real.”**

0403ED_01_

Gestión Agenda Contactos

El Código Completo

index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Agenda de Contactos</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.m
in.css" rel="stylesheet">
</head>
<body class="bg-light">

<div class="container mt-5">
  <h1 class="text-center mb-4">📒 Agenda de Contactos</h1>

  <!-- FORMULARIO -->
  <div class="card mb-4 shadow">
    <div class="card-body">
      <h5 class="card-title">Contacto</h5>

      <div class="row g-3">
        <div class="col-md-6">
          <input type="text" id="nombre" class="form-control"
placeholder="Nombre">
        </div>
        <div class="col-md-6">
          <input type="text" id="telefono" class="form-control"
placeholder="Teléfono">
        </div>
        <div class="col-md-6">
          <input type="email" id="email" class="form-control"
placeholder="Email">
        </div>
        <div class="col-md-6">
          <input type="text" id="empresa" class="form-control"
placeholder="Empresa">
        </div>
      </div>
```

0403ED_01_

Gestión Agenda Contactos

```
      <div class="mt-3 d-flex gap-2 flex-wrap">
        <button class="btn btn-primary"
onclick="agregarContacto()">Añadir (push)</button>
        <button class="btn btn-success"
onclick="agregarUrgente()">Añadir urgente (unshift)</button>
        <button class="btn btn-warning"
onclick="deshacerUltimo()">Deshacer último (pop)</button>
        <button class="btn btn-secondary"
onclick="atenderPrimero()">Atender primero (shift)</button>
      </div>
    </div>
  </div>

  <!-- TABLA -->
  <div class="card shadow">
    <div class="card-body">
      <h5 class="card-title">Listado</h5>

      <table class="table table-striped table-hover">
        <thead class="table-dark">
          <tr>
            <th>Nombre</th>
            <th>Teléfono</th>
            <th>Email</th>
            <th>Empresa</th>
            <th>Acciones</th>
          </tr>
        </thead>
        <tbody id="tablaContactos"></tbody>
      </table>
    </div>
  </div>
</div>

<script src="agenda.js"></script>
</body>
</html>
```

0403ED_01_

Gestión Agenda Contactos

agenda.js

```
// ================================
// ARRAY DE OBJETOS (BASE DE DATOS)
// ================================

const agenda = [
  {
    id: 1,
    nombre: "Ana López",
    telefono: "600123123",
    email: "ana@mail.com",
    empresa: "TechSoft"
  },
  {
    id: 2,
    nombre: "Carlos Pérez",
    telefono: "699456456",
    email: "carlos@mail.com",
    empresa: "Innova"
  }
];

// ================================
// MOSTRAR CONTACTOS (READ)
// ================================

function mostrarContactos() {
  const tbody = document.getElementById("tablaContactos");
  tbody.innerHTML = "";

  agenda.forEach(contacto => {
    tbody.innerHTML += `
      <tr>
        <td>${contacto.nombre}</td>
        <td>${contacto.telefono}</td>
        <td>${contacto.email}</td>
        <td>${contacto.empresa}</td>
        <td>
```

0403ED_01_

Gestión Agenda Contactos

```
          <button class="btn btn-danger btn-sm"
onclick="eliminarContacto(${contacto.id})">
            Eliminar
          </button>
        </td>
      </tr>
    `;
  });
}

mostrarContactos();

// ================================
// CREATE - push()
// ================================

function agregarContacto() {
  const contacto = obtenerDatosFormulario();
  if (!contacto) return;

  agenda.push(contacto); // 👉 push
  limpiarFormulario();
  mostrarContactos();
}

// ================================
// CREATE PRIORITARIO - unshift()
// ================================

function agregarUrgente() {
  const contacto = obtenerDatosFormulario();
  if (!contacto) return;

  agenda.unshift(contacto); // 👉 unshift
  limpiarFormulario();
  mostrarContactos();
}

// ================================
// DELETE ÚLTIMO - pop()
// ================================
```

0403ED_01_

Gestión Agenda Contactos

```
function deshacerUltimo() {
  if (agenda.length === 0) return;

  agenda.pop(); // 👉 pop
  mostrarContactos();
}

// ================================
// DELETE PRIMERO - shift()
// ================================

function atenderPrimero() {
  if (agenda.length === 0) return;

  agenda.shift(); // 👉 shift
  mostrarContactos();
}

// ================================
// DELETE POR ID
// ================================

function eliminarContacto(id) {
  const index = agenda.findIndex(c => c.id === id);
  if (index !== -1) {
    agenda.splice(index, 1);
    mostrarContactos();
  }
}

// ================================
// UTILIDADES
// ================================

function obtenerDatosFormulario() {
  const nombre = document.getElementById("nombre").value.trim();
  const telefono = document.getElementById("telefono").value.trim();
  const email = document.getElementById("email").value.trim();
  const empresa = document.getElementById("empresa").value.trim();

  if (!nombre || !telefono || !email || !empresa) {
    alert("Todos los campos son obligatorios");
```

0403ED_01_

Gestión Agenda Contactos

```
    return null;
  }

  return {
    id: Date.now(),
    nombre,
    telefono,
    email,
    empresa
  };
}

function limpiarFormulario() {
  document.getElementById("nombre").value = "";
  document.getElementById("telefono").value = "";
  document.getElementById("email").value = "";
  document.getElementById("empresa").value = "";
}
```

0403ED_01_

Gestión Agenda Contactos

0403ED_01_
Gestión Agenda Contactos

Anexo Bootstrap

🟦 Implementación de Bootstrap paso a paso

Agenda de Contactos

🟩 PASO 1 · Incluir Bootstrap en el proyecto

🎯 Objetivo

Poder usar **estilos profesionales** sin escribir CSS propio.

📌 Qué hacemos

Incluimos Bootstrap desde su **CDN oficial** en el `<head>` del HTML:

```
<link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
  rel="stylesheet"
>
```

🧠 Qué significa esto

- Descarga Bootstrap desde internet
- Aplica estilos globales automáticamente
- Nos permite usar **clases predefinidas**

🟩 PASO 2 · Estilo base del documento

0403ED_01_

Gestión Agenda Contactos

`<body class="bg-light">`

🎯 Objetivo

Aplicar un **color de fondo suave** a toda la página.

🧠 Clase usada

Clase	Qué hace
`bg-light`	Fondo gris claro

👉 Bootstrap tiene fondos predefinidos:
`bg-light`, `bg-dark`, `bg-primary`, etc.

🟩 PASO 3 · Contenedor principal (`container`)

`<div class="container mt-5">`

🎯 Objetivo

Centrar el contenido y limitar el ancho.

🧠 Clases usadas

Clase	Función
`container`	Ancho responsivo y centrado
`mt-5`	Margen superior grande

👉 `mt` = margin-top
👉 `5` = tamaño grande

🟩 PASO 4 · Título con utilidades Bootstrap

`<h1 class="text-center mb-4">📒 Agenda de Contactos</h1>`

0403ED_01_

Gestión Agenda Contactos

🎯 Objetivo

Centrar el título y separarlo visualmente.

🧠 Clases usadas

Clase	Función
`text-center`	Centra el texto
`mb-4`	Margen inferior

🟩 PASO 5 · Card: estructura visual principal

`<div class="card mb-4 shadow">`

🎯 Objetivo

Agrupar contenido como una **tarjeta profesional**.

🧠 Clases usadas

Clase	Función
`card`	Componente visual de tarjeta
`mb-4`	Separación inferior
`shadow`	Sombra suave

👉 `card` es uno de los componentes más usados en dashboards.

🟩 PASO 6 · Contenido interno de la card

`<div class="card-body">`

🎯 Objetivo

Añadir **padding interno automático**.

0403ED_01_

Gestión Agenda Contactos

👉 Sin `card-body`, el contenido queda pegado a los bordes.

🟩 PASO 7 · Sistema de grid (row + col)

```
<div class="row g-3">
```

🎯 Objetivo

Crear un layout **responsivo por columnas**.

🧠 Clases usadas

Clase	Función
`row`	Fila del grid
`g-3`	Espaciado entre columnas

📌 Columnas

```
<div class="col-md-6">
```

🧠 Qué significa

Parte	Significado
`col`	Columna
`md`	A partir de tamaño medio
`6`	6 de 12 columnas

👉 En pantallas grandes: 2 columnas
👉 En móvil: 1 columna automática

🟩 PASO 8 · Inputs con Bootstrap

```
<input type="text" class="form-control">
```

0403ED_01_

Gestión Agenda Contactos

🎯 Objetivo

Estilos uniformes y accesibles.

🧠 Clase usada

Clase	Función
`form-control`	Estilo profesional para inputs

👉 Incluye:

- padding
- borde
- foco accesible
- ancho completo

🟩 PASO 9 · Botones Bootstrap

```
<button class="btn btn-primary">Añadir</button>
```

🎯 Objetivo

Botones claros y consistentes.

🧠 Clases usadas

Clase	Función
`btn`	Botón base
`btn-primary`	Color principal

Otros usados:

- `btn-success`
- `btn-warning`
- `btn-danger`
- `btn-secondary`

0403ED_01_

Gestión Agenda Contactos

🟩 PASO 10 · Agrupar botones (flex utilities)

`<div class="mt-3 d-flex gap-2 flex-wrap">`

🎯 Objetivo

Alinear botones correctamente.

🧠 Clases usadas

Clase	Función
`d-flex`	Activa Flexbox
`gap-2`	Espacio entre botones
`flex-wrap`	Permite salto de línea
`mt-3`	Margen superior

🟩 PASO 11 · Tabla Bootstrap

`<table class="table table-striped table-hover">`

🎯 Objetivo

Mostrar datos tabulares claros.

🧠 Clases usadas

Clase	Función
`table`	Estilo base
`table-striped`	Filas alternas
`table-hover`	Efecto hover

📌 Cabecera oscura

0403ED_01_

Gestión Agenda Contactos

`<thead class="table-dark">`

🎯 Objetivo

Diferenciar encabezado del contenido.

🟩 PASO 12 · Botones pequeños en la tabla

`<button class="btn btn-danger btn-sm">Eliminar</button>`

🧠 Clase extra

Clase	Función
`btn-sm`	Botón pequeño

🟦 RESUMEN GLOBAL

✔ Componentes Bootstrap usados

- container
- card
- grid (row / col)
- form-control
- buttons
- table

✔ Utilidades usadas

- spacing (`mt`, `mb`, `gap`)
- flex (`d-flex`, `flex-wrap`)
- colores (`bg-light`, `table-dark`)
- tipografía (`text-center`)

🎯 Frase clave para el alumno

0403ED_01_

Gestión Agenda Contactos

Bootstrap no es magia

Es una colección de clases que:

- ahorran tiempo
- garantizan consistencia
- hacen el diseño responsive

0403ED_01_

Gestión Agenda Contactos

🔍 Responsive Breakpoints en Bootstrap

Explicación paso a paso (con ejemplos reales)

🧠 ¿Qué es un breakpoint?

Un **breakpoint** es un **punto de ruptura del ancho de pantalla** donde el diseño **cambia su comportamiento**.

👉 En Bootstrap, los breakpoints se usan para decir:

> "A partir de este tamaño de pantalla, el layout cambia"

🟩 Breakpoints oficiales de Bootstrap 5

Nombre	Prefijo	Ancho mínimo
Extra pequeño	*(sin prefijo)*	`< 576px`
Small	`sm`	`≥ 576px`
Medium	`md`	`≥ 768px`
Large	`lg`	`≥ 992px`
Extra Large	`xl`	`≥ 1200px`
XXL	`xxl`	`≥ 1400px`

📌 Bootstrap es **mobile first**:

- Sin prefijo → móvil
- Con prefijo → pantallas más grandes

🟩 Regla de oro (MUY IMPORTANTE)

`col-md-6`

0403ED_01_
Gestión Agenda Contactos

❌ NO significa “solo en md”
✅ Significa:

> “Desde md en adelante”

🟩 Ejemplo clave de tu proyecto

Código usado en la agenda

```
<div class="col-md-6">
```

Comportamiento real

Pantalla	Resultado
📱 Móvil (<768px)	1 columna (100%)
💻 Tablet / PC (≥768px)	2 columnas (50% + 50%)

👉 **Sin escribir media queries**.

🟩 PASO 1 · Mobile First (sin prefijo)

```
<div class="col-12">
```

🧠 Qué significa

- Ocupa todo el ancho
- Ideal para móvil

📌 En Bootstrap, **si no pones nada**, ya es mobile first.

🟩 PASO 2 · Añadir comportamiento en `md`

```
<div class="col-12 col-md-6">
```

0403ED_01_

Gestión Agenda Contactos

🧠 Qué ocurre

- Móvil → 1 columna
- Tablet / PC → 2 columnas

👉 Es el patrón más usado en proyectos reales.

🟩 PASO 3 · Varios breakpoints combinados

```
<div class="col-12 col-md-6 col-lg-4">
```

🧠 Comportamiento

Pantalla	Columnas
📱 Móvil	1
📱 Tablet	2
💻 PC grande	3

🟩 PASO 4 · Breakpoints en botones (flex)

```
<div class="d-flex flex-column flex-md-row">
```

🧠 Qué pasa

Pantalla	Resultado
📱 Móvil	Botones en columna
💻 Desde md	Botones en fila

👉 Esto **lo estás usando indirectamente**.

🟩 PASO 5 · Ocultar / mostrar según tamaño

```
<div class="d-none d-md-block">
```

0403ED_01_
Gestión Agenda Contactos

🧠 Comportamiento

- Móvil → oculto
- Desde md → visible

```
<div class="d-block d-md-none">
```

- Visible solo en móvil

🟩 PASO 6 · Breakpoints en spacing

```
<div class="mt-2 mt-md-4">
```

🧠 Qué hace

- Móvil → poco margen
- Pantallas grandes → más aire

🟩 PASO 7 · Breakpoints en texto

```
<h1 class="text-center text-md-start">
```

🧠 Resultado

- Móvil → centrado
- Desktop → alineado a la izquierda

🟩 PASO 8 · Aplicación REAL en tu agenda

Inputs

```
<div class="col-12 col-md-6">
```

0403ED_01_

Gestión Agenda Contactos

✔ Móvil: formulario vertical
✔ Desktop: formulario en dos columnas

Tabla

📱 En móvil:

- Se hace scroll horizontal
- Mantiene legibilidad

💻 En desktop:

- Tabla completa

(Si quisieras mejorar móvil: `table-responsive`)

```
<div class="table-responsive">
```

🧠 Diferencia con CSS clásico

CSS puro

```
@media (min-width: 768px) {
  .campo {
    width: 50%;
  }
}
```

Bootstrap

```
<div class="col-md-6">
```

✔ Más rápido
✔ Más legible
✔ Menos CSS

🟦 RESUMEN CLAVE (EXAMEN / ENTREVISTA)

0403ED_01_

Gestión Agenda Contactos

📌 Frases que valen oro

- Bootstrap es **mobile first**
- Los breakpoints son **mínimos**
- `col-md-6` = “desde md en adelante”
- Sin prefijo = móvil
- Se pueden combinar breakpoints

🎯 Mini ejercicio para el alumno

👉 Modifica el formulario para que:

1. En móvil → 1 columna
2. En tablet → 2 columnas
3. En desktop → 3 columnas

💡 Pista:

```
<div class="col-12 col-md-6 col-lg-4">
```

0403ED_01_
Gestión Agenda Contactos

🧩 Agenda Responsive con CSS puro + Media Queries

📁 Estructura del proyecto

```
agenda-css/
 ├── index.html
 ├── styles.css
 └── agenda.js
```

(`agenda.js` es el MISMO que ya usas, no cambia la lógica)

1️⃣ `index.html` (SIN Bootstrap)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Agenda de Contactos</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

<div class="container">
  <h1>📒 Agenda de Contactos</h1>

  <!-- FORMULARIO -->
  <div class="card">
    <h2>Contacto</h2>

    <div class="form-grid">
      <input type="text" id="nombre" placeholder="Nombre">
      <input type="text" id="telefono" placeholder="Teléfono">
```

```
      <input type="email" id="email" placeholder="Email">
      <input type="text" id="empresa" placeholder="Empresa">
    </div>

    <div class="actions">
      <button class="primary"
onclick="agregarContacto()">Añadir</button>
      <button class="success"
onclick="agregarUrgente()">Urgente</button>
      <button class="warning"
onclick="deshacerUltimo()">Deshacer</button>
      <button class="secondary"
onclick="atenderPrimero()">Atender</button>
    </div>
  </div>

  <!-- TABLA -->
  <div class="card">
    <h2>Listado</h2>

    <div class="table-wrapper">
      <table>
        <thead>
          <tr>
            <th>Nombre</th>
            <th>Teléfono</th>
            <th>Email</th>
            <th>Empresa</th>
            <th>Acciones</th>
          </tr>
        </thead>
        <tbody id="tablaContactos"></tbody>
      </table>
    </div>
  </div>
</div>

<script src="agenda.js"></script>
</body>
</html>
```

2️⃣ `styles.css` (CSS PURO + RESPONSIVE)

🟩 PASO 1 · Reset y estilos base

```
* {
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

body {
  margin: 0;
  background: #f2f2f2;
}
```

🔍 **Qué sustituye de Bootstrap**

- Normalización
- Tipografía base
- Fondo `bg-light`

🟩 PASO 2 · Contenedor central

```
.container {
  max-width: 1100px;
  margin: 40px auto;
  padding: 20px;
}
```

🔍 Equivalente a:

```
<div class="container mt-5">
```

0403ED_01_

Gestión Agenda Contactos

🟩 PASO 3 · Cards (tarjetas)

```
.card {
  background: white;
  padding: 20px;
  border-radius: 6px;
  margin-bottom: 30px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
```

🔍 Equivalente a:

- `card`
- `card-body`
- `shadow`

🟩 PASO 4 · Grid del formulario (mobile first)

```
.form-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 15px;
}
```

📱 **Por defecto (móvil)**
→ 1 columna

🟩 PASO 5 · Inputs (form-control)

```
input {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
  width: 100%;
}
```

🔍 Equivalente a `form-control`

0403ED_01_

Gestión Agenda Contactos

🟩 PASO 6 · Botones (btn + variantes)

```
.actions {
  margin-top: 20px;
  display: flex;
  flex-wrap: wrap;
  gap: 10px;
}

button {
  padding: 10px 16px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  color: white;
}

.primary { background: #0d6efd; }
.success { background: #198754; }
.warning { background: #ffc107; color: black; }
.secondary { background: #6c757d; }
```

🔍 Sustituye:

- `btn`
- `btn-primary`
- `btn-success`
- `gap`
- `flex-wrap`

🟩 PASO 7 · Tabla base

```
.table-wrapper {
  overflow-x: auto;
}

table {
  width: 100%;
```

```
  border-collapse: collapse;
}

thead {
  background: #212529;
  color: white;
}

th, td {
  padding: 10px;
  border-bottom: 1px solid #ddd;
  text-align: left;
}
```

🔍 Sustituye:

- `table`
- `table-striped`
- `table-dark`
- `table-responsive`

🟩 PASO 8 · Responsive con MEDIA QUERIES

📱 Tablet (≥ 768px)

```
@media (min-width: 768px) {
  .form-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

📌 Resultado:

- móvil → 1 columna
- tablet → 2 columnas

💻 Desktop (≥ 1024px)

```
@media (min-width: 1024px) {
```

0403ED_01_

Gestión Agenda Contactos

```
  .form-grid {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

📌 Resultado:

- desktop → 3 columnas

🧠 Comparación mental Bootstrap vs CSS puro

Bootstrap	CSS puro
`container`	`.container`
`row` / `col-md-6`	`grid` + `media queries`
`card`	`.card`
`btn` `btn-primary`	`.primary`
`table-responsive`	`overflow-x: auto`
breakpoints	`@media (min-width)`

🎯 Resumen docente (clave)

- Bootstrap **no hace magia**
- Todo se basa en:
  - Flexbox
  - Grid
  - Media Queries
- Entender CSS puro = dominar cualquier framework

🧪 Mini reto para el alumno

0403ED_01_

Gestión Agenda Contactos

1. Cambia el breakpoint desktop a `1200px`
2. Haz que los botones:
    - móvil → columna
    - desktop → fila

💡 Pista:

```
@media (min-width: 768px) {
  .actions {
    flex-direction: row;
  }
}
```